

Autocomplete and Autocorrect using NLP

Oasis Infobyte

Track: Data Analytics

Task: Task 5 – Autocomplete and Autocorrect using NLP

Author: Bhavyaa Bansal

Abstract

This project implements two classical Natural Language Processing (NLP) applications: autocomplete and autocorrect using a Wikipedia text corpus. A frequency-based bigram language model predicts the next word, while spelling correction is implemented using both PySpellChecker and a custom Levenshtein Distance algorithm. The project compares both approaches using accuracy, precision and recall, and discusses their strengths and limitations.

Methodology

- Collected a large Wikipedia text corpus.
 - Performed preprocessing including lowercasing, punctuation removal, tokenization and stopword removal.
 - Built a bigram frequency model for autocomplete.
 - Generated Top-3 predictions for multiple input prefixes.
 - Implemented autocorrect using PySpellChecker and a custom Levenshtein Distance algorithm.
 - Evaluated the approaches using test words, accuracy, precision, recall and visualizations.
-

Results

Autocomplete successfully generated context-based next-word suggestions using word frequency information from the corpus. PySpellChecker achieved approximately 95% accuracy on the prepared misspelled-word dataset, while the custom Levenshtein implementation achieved approximately 65% accuracy. The difference highlights the benefit of optimized dictionaries and frequency-aware candidate ranking.

Discussion

The project demonstrates the effectiveness of classical NLP techniques for text prediction and spelling correction. However, the bigram model considers only the previous word and cannot understand sentence context. Similarly, the custom spell corrector relies solely on edit distance and may return incorrect words when the corpus contains spelling mistakes. Production systems such as Google Keyboard use transformer-based language models, contextual understanding, personalization and large-scale language statistics to provide more accurate predictions.

Conclusion

The project successfully fulfilled the objectives of implementing and evaluating autocomplete and autocorrect algorithms using NLP. It provides practical experience with language modelling, edit-distance algorithms, text preprocessing and performance evaluation, while demonstrating the differences between classical NLP techniques and modern intelligent typing assistants.